MANUAL TECNICO

Juan Pablo de León Miranda

Quetzaltenango, febrero de 2026

Contenido

Analizadores 3
Lexer.jflex 3
Parser.cup 4
Gramatica Sintactica 5
Expression 8
Aritmetica 8
Ejecutable 8
OperacionMathNoRetornable 10
Native 11
Flujos – SI y MIENTRAS 12
Instrucción 12
Reportes – Aritmeticos y Control 12

Analizadores

Lexer.jflex

Consiste en convertir los elementos de entrada en tokens que serán usados para validación del lenguaje.

Contiene dos listas:

1. LinkedList<SintaxError> listaErrorLexico dedicada a almacenar errores de tipo léxico
2. LinkedList<Token> listaTokens dedicada a almacenar todos los elementos que sean considerados tokens validos en el lenguaje definido

Se encuentra también una gramática definida, donde nos enfocaremos en las siguientes expresiones regulares

- **NUMEROS = [0-9]+ :** Esta nos permite valores de números enteros
- **DEFINICION_ID = [a-zA-Z0-9_]+ :** esta nos permite guardar secuencia de letras, números o guiones bajos, esta creada para poder almacenar los nombres de los identificadores
- **LETRAS = [a-zA-Z]+ :** nos permite agrupar una secuencia de letras (siendo no case sensitive)
- **DECIMAL = {NUMEROS}"."{NUMEROS} :** nos permite reconocer números con formato decimal, ya que permite una interrupción por medio de un punto
- **IDENTIFICADOR = ({LETRAS}|"_")+({LETRAS}|{NUMEROS}|"_")*** : es para poder hacer valida la estructura de un identificador, haciendo de que de inicio no puedan haber números
- **HEXADECIMAL = "H"[0-9A-F]{6} :** nos permite verificar valores hexadecimales, es decir números del 0al 9 y letras mayúsculas de A a F con un total definido de 6 elementos exactos por hexadecimal
- **TEXTO = \"[^\"]*\"** : Nos permite validar cualquier carácter dentro de dos comillas dobles.
- **espacios = [\t\r\n\f]+ :** nos permite reconocer los espacios, saltos de línea, retorno de carro y tabulación una o mas veces
- **SEPARADOR = "%%%%" :** elemento que sirve como separador entre el código a graficar y las configuraciones
- **"#".* :** nos permite aceptar comentarios de una sola línea, es decir cualquier carácter donde antes este la almoadilla será considerado valido y no afectara en el reconocimiento de caracteres fuera del lenguaje

- Por ultimo en la siguiente imagen se encuentran todas las palabras reservadas, esdecir aqueas que siguen estrictamente un orden para poder ser validas

```
/******PALABRAS RESERVADAS********/

//Pseudocodigo
<YYINITIAL> INICIO { agregarToken(yytext(), "INICIO"); return new Symbol(sym.INICIO, yyline, yycolumn, yytext());}
<YYINITIAL> FIN { agregarToken(yytext(), "FIN"); return new Symbol(sym.FIN, yyline, yycolumn, yytext());}

<YYINITIAL> SI { agregarToken(yytext(), "SI"); return new Symbol(sym.SI, yyline, yycolumn, yytext());}
<YYINITIAL> ENTONCES { agregarToken(yytext(), "ENTONCES"); return new Symbol(sym.ENTONCES, yyline, yycolumn, yytext());}
<YYINITIAL> FINSI { agregarToken(yytext(), "FINSI"); return new Symbol(sym.FINSI, yyline, yycolumn, yytext());}

<YYINITIAL> MIENTRAS { agregarToken(yytext(), "MIENTRAS"); return new Symbol(sym.MIENTRAS, yyline, yycolumn, yytext());}
<YYINITIAL> HACER { agregarToken(yytext(), "HACER"); return new Symbol(sym.HACER, yyline, yycolumn, yytext());}
<YYINITIAL> FINMIENTRAS { agregarToken(yytext(), "FINMIENTRAS"); return new Symbol(sym.FINMIENTRAS, yyline, yycolumn, yytext());}

<YYINITIAL> MOSTRAR { agregarToken(yytext(), "MOSTRAR"); return new Symbol(sym.MOSTRAR, yyline, yycolumn, yytext());}

<YYINITIAL> LEER { agregarToken(yytext(), "LEER"); return new Symbol(sym.LEER, yyline, yycolumn, yytext());}

<YYINITIAL> VAR { agregarToken(yytext(), "VAR"); return new Symbol(sym.VAR, yyline, yycolumn, yytext());}

//Configuraciones
<YYINITIAL> %DEFAULT { agregarToken(yytext(), "DEFAULT"); return new Symbol(sym.DEFAULT, yyline, yycolumn, yytext());}

<YYINITIAL> %COLOR_TEXTO_SI { agregarToken(yytext(), "COLOR_TEXTO_SI"); return new Symbol(sym.COLOR_TEXTO_SI, yyline, yycolumn, yytext());}
<YYINITIAL> %COLOR_SI { agregarToken(yytext(), "COLOR_SI"); return new Symbol(sym.COLOR_SI, yyline, yycolumn, yytext());}
<YYINITIAL> %FIGURA_SI { agregarToken(yytext(), "FIGURA_SI"); return new Symbol(sym.FIGURA_SI, yyline, yycolumn, yytext());}
<YYINITIAL> %LETRAS_SI { agregarToken(yytext(), "LETRAS_SI"); return new Symbol(sym.LETRAS_SI, yyline, yycolumn, yytext());}
<YYINITIAL> %LETRAS_SIZE_SI { agregarToken(yytext(), "LETRAS_SIZE_SI"); return new Symbol(sym.LETRAS_SIZE_SI, yyline, yycolumn, yytext());}

<YYINITIAL> %COLOR_TEXTO_MIENTRAS { agregarToken(yytext(), "COLOR_TEXTO_MIENTRAS"); return new Symbol(sym.COLOR_TEXTO_MIENTRAS, yyline, yycolumn, yytext());}
<YYINITIAL> %COLOR_MIENTRAS { agregarToken(yytext(), "COLOR_MIENTRAS"); return new Symbol(sym.COLOR_MIENTRAS, yyline, yycolumn, yytext());}
<YYINITIAL> %FIGURA_MIENTRAS { agregarToken(yytext(), "FIGURA_MIENTRAS"); return new Symbol(sym.FIGURA_MIENTRAS, yyline, yycolumn, yytext());}
<YYINITIAL> %LETRAS_MIENTRAS { agregarToken(yytext(), "LETRAS_MIENTRAS"); return new Symbol(sym.LETRAS_MIENTRAS, yyline, yycolumn, yytext());}
<YYINITIAL> %LETRAS_SIZE_MIENTRAS { agregarToken(yytext(), "LETRAS_SIZE_MIENTRAS"); return new Symbol(sym.LETRAS_SIZE_MIENTRAS, yyline, yycolumn, yytext());}

<YYINITIAL> %COLOR_TEXTO_BLOQUE { agregarToken(yytext(), "COLOR_TEXTO_BLOQUE"); return new Symbol(sym.COLOR_TEXTO_BLOQUE, yyline, yycolumn, yytext());}
<YYINITIAL> %COLOR_BLOQUE { agregarToken(yytext(), "COLOR_BLOQUE"); return new Symbol(sym.COLOR_BLOQUE, yyline, yycolumn, yytext());}
<YYINITIAL> %FIGURA_BLOQUE { agregarToken(yytext(), "FIGURA_BLOQUE"); return new Symbol(sym.FIGURA_BLOQUE, yyline, yycolumn, yytext());}
<YYINITIAL> %LETRAS_BLOQUE { agregarToken(yytext(), "LETRAS_BLOQUE"); return new Symbol(sym.LETRAS_BLOQUE, yyline, yycolumn, yytext());}
```

```
<YYINITIAL> %LETRAS_SIZE_BLOQUE { agregarToken(yytext(), "LETRAS_SIZE_BLOQUE"); return new Symbol(sym.LETRAS_SIZE_BLOQUE, yyline, yycolumn, yytext());}

//Tipos de Forma
<YYINITIAL> ECLIPSE { agregarToken(yytext(), "ECLIPSE"); return new Symbol(sym.ECLIPSE, yyline, yycolumn, yytext());}

<YYINITIAL> CIRCULO { agregarToken(yytext(), "CIRCULO"); return new Symbol(sym.CIRCULO, yyline, yycolumn, yytext());}

<YYINITIAL> PARALELOGRAMO { agregarToken(yytext(), "PARALELOGRAMO"); return new Symbol(sym.PARALELOGRAMO, yyline, yycolumn, yytext());}

<YYINITIAL> RECTANGULO { agregarToken(yytext(), "RECTANGULO"); return new Symbol(sym.RECTANGULO, yyline, yycolumn, yytext());}

<YYINITIAL> ROMBO { agregarToken(yytext(), "ROMBO"); return new Symbol(sym.ROMBO, yyline, yycolumn, yytext());}

<YYINITIAL> RECTANGULO_REDONDEADO { agregarToken(yytext(), "RECTANGULO_REDONDEADO"); return new Symbol(sym.RECTANGULO_REDONDEADO, yyline, yycolumn, yytext());}

//Tipos de Letra
<YYINITIAL> ARIAL { agregarToken(yytext(), "ARIAL"); return new Symbol(sym.ARIAL, yyline, yycolumn, yytext());}

<YYINITIAL> TIMES_NEW_ROMAN { agregarToken(yytext(), "TIMES_NEW_ROMAN"); return new Symbol(sym.TIMES_NEW_ROMAN, yyline, yycolumn, yytext());}

<YYINITIAL> COMIC_SANS { agregarToken(yytext(), "COMIC_SANS"); return new Symbol(sym.COMIC_SANS, yyline, yycolumn, yytext());}

<YYINITIAL> VERDANA { agregarToken(yytext(), "VERDANA"); return new Symbol(sym.VERDANA, yyline, yycolumn, yytext());}
```

Parser.cup

Esta destinado para el análisis sintactico del programa, este recibe los tokens leídos por Lexer y verifica que la estructura de estos sea la adecuada.

Podemos encontrar de inicio una serie de código que nos permite el control y agrupación de errores sintácticos encontrados al momento de ser llamados por Parser

```
//Listas para guardar Errores sintacticos
Parser s;
Parser(Parser s){this.s = s;}

public LinkedList<SintaxError> listErrorSintactico = new LinkedList<>();

public void syntax_error(Symbol s){
    listErrorSintactico.add(new SintaxError("Error de sintaxis recuperable", "No se esperaba: " +s.value, s.left, s.right));
}

public void unrecovered_syntax_(Symbol s) throws java.lang.Exception{
    listErrorSintactico.add(new SintaxError("Error de sintaxis no recuperable", "No se esperaba: " +s.value, s.left, s.right));
}
```

Gramatica Sintactica

Iniciamos con

```
start with iniciacion;

iniciacion ::= instrucciones:a {: RESULT = a; :}
             ;
```

esto nos retorna a

```
instrucciones ::= instrucciones:a instruccion:b {: RESULT = a; RESULT.add(b); :}
                | instrucciones:a inicio:b {: RESULT  = a; RESULT.addAll(b); :}
                | instruccion:a {: RESULT = new LinkedList<Instruccion>(); RESULT.add(a); :}
                | inicio:a {: RESULT = a; :}
                ;
```

Donde podremos tener varios caminos:

1. Nos vuelve a retornar a instrucciones y nos va agregando los valores a la lista Instrucción
2. Nos redirige al formato de nuestro lenguaje
3. Nos inicializa la lista Instrucción y nos va agregando los valores

Dentro de nuestra gramática inicial podemos ver 5 partes

```
//adimite unicamente la estructura INICIO "codigo" FIN %%%% "configuraciones para graficacion"
inicio ::= INICIO codigo_start:a FIN SEPARADOR configuracion_personalizada:b {: RESULT = a; :}
         ;
```

Las cuales son:

1. INICIO : palabra necesaria para poder iniciar el programa
2. codigo_start : nos redirige hacias las demás instrucciones validas de nuestro lenguaje
3. FIN : nos indica que todo el código entre INICIO y FIN es el analizado

4. SEPARADOR : es el limitante entre código y configuraciones
5. congifuracion_personalizada : nos redirige hacia donde se encuentra la forma de personalizar nuestros diagramas

Dentro de código_start nos hará poder repetir las diferentes instrucciones mas de una vez

```
codigo_start ::= codigo_start:a codigo:b {: a.add(b);
                                            RESULT = a; :} //anida la lista nueva con la inicial
                                            //crea lista nueva para poder guardar/repetir los bloques de codigo
                | codigo:a {: LinkedList<Instruccion> lista = new LinkedList<Instruccion>();
                              lista.add(a);
                              RESULT = lista; :} //devuelve la nueva lista
                ;
```

Dentro de código se encuentran los diferentes caminos hacia el bloque de código, la estructura SI o MIENTRAS

```
//puede admitir un bloque de codigo, la estructura SI o MIENTRAS al inicio y repetirlo en cualquierm momento
codigo ::= bloque:a {: RESULT = a; :}
         | estruc_si:a {: RESULT = a; :}
         | estruc_mientras:a {: RESULT = a; :}
```

Dentro de bloque nos mostrara el camino hacia poder crear variables, o el uso de las palabras MOSTRAR y LEER

```
//bloque esta consituido por las variables, MOSTRAR y LEER
bloque ::= variable:a {: RESULT = a; :}
         | estruc_mostrar:a {: RESULT = a; :}
         | estruc_leer:a {: RESULT = a; :}
         ;

//variable admite iniciar por VAR o directamente por el nombre
//sino tiene VAR al inicio debe de asignar un valor/operacion
variable ::=  VAR IDENTIFICADOR:a ASIGNACION op_math_bloque_1:b {: RESULT = new Declaracion((String) a, b, aleft, aright); :}
            | VAR IDENTIFICADOR:a {: RESULT = new Declaracion((String) a, null, aleft, aright); :}
            | IDENTIFICADOR:a ASIGNACION op_math_bloque_1:b {: RESULT = new Declaracion((String) a, b, aleft, aright); :}
            ;

//admite texto dentro de comillas o el nombre de una variable
estruc_mostrar ::= MOSTRAR TEXTO:a {: RESULT = new Native(a,new Type(TypeData.STRING), aleft, aright); :}
                 | MOSTRAR IDENTIFICADOR:a {: RESULT = new AccesVariable((String) a, aleft, aright); :}
                 ;

//admite unicament el nombre de una variable
estruc_leer ::= LEER IDENTIFICADOR:a {: RESULT = new Native(a,new Type(TypeData.STRING), aleft, aright); :}
```

La parte de las estructuras SI y MIENTRAS se toman juntas ya que en esta gramática siguen ambos un mismo patron

```
//esta constituida por SI "una condicion logica o relacional" ENTONCES "bloque de codigo cualquiera" FINSI/FIN SI
estruc_si ::= SI condicion:a ENTONCES codigo_start:b estruc_fin_si {: RESULT = new Estructuira_SI(a, b, aleft, aright); :}
            ;

//admite terminal el bloque con las palabras FINSI o FIN SI
estruc_fin_si ::= FINSI
                | FIN SI
                ;
esta constituida por MIENRTAS "una condicion logica o relacional" HACER "bloque de codigo cualquiera" FINMIENTRAS/FIN MIENTRAS
estruc_mientras ::= MIENTRAS condicion:a HACER codigo_start:b estruc_fin_mientras {: RESULT = new Estructura_Mientras(a, b, aleft, aright); :}
                  ;

//admite terminal el bloque con las palabras FINMIENTRAS o FIN MIENTRAS
estruc_fin_mientras ::= FINMIENTRAS
                  | FIN MIENTRAS
                  ;
```

En la parte de condición de ambas podremos ver los caminos a seguir de forma jerárquica (menor a mayor precedencia)

- ! – Nivel 1
- && y || - Nivel 2
- ==, !=, >=, <=, >, < - Nivel 3
- () – Nivel 4

```
//admite tener el simbolo ! al inicio de una condicion o usar la condicion con o sin parentesis
condicion ::= NOT condicion_compleja:a {: RESULT = a; :}
            | condicion_compleja:a {: RESULT = a; :}
            ;

//en mejor jerarquia se encuentra && y ||
condicion_compleja ::= condicion_compleja:a AND expresion_relacional:b {: RESULT = new OperacionMathNoRetornable( a, b,"&&", aleft, aright); :}
                     | condicion_compleja:a OR expresion_relacional:b {: RESULT = new OperacionMathNoRetornable( a, b, "||", aleft, aright); :}
                     | expresion_relacional:a {: RESULT = a; :}
                     ;

se encuentran todos en misma jerarquia
expresion_relacional ::= expresion_relacional:a IGUALDAD expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b,"==", aleft, aright); :}
                       | expresion_relacional:a DIFERENTE expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b, "!=", aleft, aright); :}
                       | expresion_relacional:a MAYOR_IGUAL_QUE expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b, ">=", aleft, aright); :}
                       | expresion_relacional:a MENOR_IGUAL_QUE expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b, "<=", aleft, aright); :}
                       | expresion_relacional:a MAYOR_QUE expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b, ">", aleft, aright); :}
                       | expresion_relacional:a MENOR_QUE expresion_paren:b {: RESULT = new OperacionMathNoRetornable( a, b, "<", aleft, aright); :}
                       | expresion_paren:a {: RESULT = a; :}
                       ;

//en caso de obtener parentesis regresa desde la negacion
expresion_paren ::= LPAREN condicion:a RPAREN {: RESULT = a; :}
                  | valores_variable:a {: RESULT = a; :} //sino es otra operacion admite nombre de id o numeros
                  ;
```

En la parte de las operaciones matemáticas también se mostrara de forma jerárquica

- - = Uno menos, nivel 1
- + y – = Nivel 1
- * y / = Nivel 2
- () = Nivel 3

```
//en caso de obtener parentesis regresa desde la negacion
expresion_paren ::= LPAREN condicion:a RPAREN {: RESULT = a; :}
                  | valores_variable:a {: RESULT = a; :} //sino es otra operacion admite nombre de id o numeros
                  ;

//realiza operaciones aritmeticas teniendo como prioridad cualquier operacion negativa
op_math_bloque_1 ::= RESTA op_math_bloque_1:a {: RESULT = new Negacion(a, aleft, aright); :} %prec UMENOS /* establece UMENOS como prioridad*/
                    /*suma y resta tienen menor prioridad*/
                  | op_math_bloque_1:a SUMA op_math_bloque_2:b {: RESULT = new OperacionMathNoRetornable( a, b, "+", aleft, aright); :}
                  | op_math_bloque_1:a RESTA op_math_bloque_2:b  {: RESULT = new OperacionMathNoRetornable(a, b, "-", aleft, aright); :}
                  | op_math_bloque_2:a {: RESULT = a; :}
                ;

//segundos en jerarquia esta la multiplicacion y division
op_math_bloque_2 ::= op_math_bloque_2:a MULTIPLICACION op_math_bloque_3:b {: RESULT = new OperacionMathNoRetornable(a, b, "*", aleft, aright); :}
                   | op_math_bloque_2:a DIVISION op_math_bloque_3:b {: RESULT = new OperacionMathNoRetornable(a, b, "/", aleft, aright); :}
                   | op_math_bloque_3:a {: RESULT = a; :}
                   ;
//el primero en prioridad informacion dentro del parenesis
op_math_bloque_3 ::= LPAREN op_math_bloque_1:a RPAREN {: RESULT = a; :}
                    | valores_variable:a {: RESULT = a; :}
                    ;

//busca si el nombre del idenficador ya se encuentra en el ast
//si se encuentra ingresado almacena si el elemento es un numero o el nombre del id
valores_variable ::= IDENTIFICADOR:a {: RESULT = new AccesVariable((String) a, aleft, aright); :}
                |DECIMAL:a {: RESULT = new Native(Double.parseDouble(a.toString()), new Type(TypeData.DECIMAL), aleft, aright); :}
                | ENTEROS:a {: RESULT = new Native(Integer.parseInt(a.toString()), new Type(TypeData.ENTERO), aleft, aright); :}
                | DEFINICION_ID:a {: RESULT = new Native(a,new Type(TypeData.STRING), aleft, aright); :} /*guarda unicamente el nombre del id*/
                ;
```

Expression

Aritmetica

Ejecutable

Suma, Resta, Multiplicacion, Division

Dentro de las operaciones aritméticas ejecutables contamos con la operación de Suma, Resta, Multiplicacion, Division

En esta parte únicamente hablaremos de la parte Suma, ya que las demás estructuras siguen los mismos pasos únicamente cambiando el operador de retorno

Estas clases van a heredar de Instrucción para poder ser agregadas al árbol AST, recibe 2 valores de tipo instrucción que serán los claves para poder realizar las comparación y la operación aritmética correspondiente, eso si los castea según sea uno diferente

El cambio para las demás clases se encuentra en el operador en los respectivos retornos

```
26      override fun interprete(tree: Tree, table: TableSymbol): Any? {
27          if (rightValue is Error)
28              return rightValue
29
30          //se obtiene el tipo de las instrucciones para poder verificar si se castea o unicamente realiza la accion
31          var leftType = left.typeValue.typeData
32          var rightType = right.typeValue.typeData
33
34          when (leftType){
35              TypeData.ENTERO -> {
36                  when (rightType){
37                      TypeData.ENTERO -> {//en caso de ser el valor izquierdo y derecho enteros unicamente se suman y se devuelve
38                          this.typeValue = Type(TypeData.ENTERO)
39                          return leftValue as Int + rightValue as Int
40                      }
41
42                      TypeData.DECIMAL ->{//si el segundo elemento es decimal se castea el primero a decimal y se devuelve
43                          this.typeValue = Type(TypeData.DECIMAL)
44                          if (leftValue is Number && rightValue is Number){
45                              return  (leftValue as Number).toDouble() + (rightValue as Number).toDouble()
46                          } else {
47                              return SintaxError( tipo = "SINTACTICO", mensaje = "No se pueden sumar valores no numericos", this.linea, this.columna)
48                          }
49                      }
50                      else ->{
51                          val error : SintaxError =
52                              SintaxError( tipo = "SINTACTICO", mensaje = "Suma Invalida", this.linea, this.columna)
53                          return  error
54                      }
55                  }
56              }
57
58              TypeData.DECIMAL ->{//si el primero es decimal
59                  when (rightType){
60                      TypeData.ENTERO ->{//si el segundo es decimal se casean para manejar un mismo valor
61                          this.typeValue = Type(TypeData.DECIMAL)//cambia el valor a tipo decimal
62                          return (leftValue as Double) + (rightValue as Double)
63                      }
64                      TypeData.DECIMAL ->{ //ambos decimales unicamente suma
65                          this.typeValue = Type(TypeData.DECIMAL)
66                          return  leftValue as Double + rightValue as Double
67                      }
68                      else -> {
69                          val error : SintaxError =
70                              SintaxError( tipo = "SINTACTICO", mensaje = "Suma Invalida", this.linea, this.columna)
71                          return  error
72                      }
73                  }
74                  else -> {
75                      val error : SintaxError =
76                          SintaxError( tipo = "SINTACTICO", mensaje = "Suma Invalida", this.linea, this.columna)
77                      return  error
78                  }
```

OperacionReportes

Es una clase de tipo objeto que almacena las operaciones aritméticas con formato hacia los reportes aritméticos, recibe el tipo del operador en forma de string, además de el valor de los números, su tipo.

Dentro de las operaciones castea todo hacia double y verifica si inicialmente ambos eran de tipo entero para asi devolverlo igual, en caso de que no lo sea los devuelve como double

```
2 Usages
object OperacionesReportes {
    //unica clase a la que se puede acceder
    1 Usage
    fun calcular(
        leftValue: Any?, //el valor izquierdo
        leftType: TypeData?, //el tipo del valor izquierdo
        rightValue: Any?,// el valor derecho
        rightType: TypeData?,// el tipo del valor derecho
        operador: String, //el tipo de operador
        linea: Int,
        columna: Int
    ): Any? {
        return when (operador) { //dependiendo del operador admitido realiza la operacion
            "+" -> suma(leftValue, leftType, rightValue, rightType, linea, columna)
            "-" -> resta(leftValue, leftType, rightValue, rightType, linea, columna)
            "*" -> multiplicacion(leftValue, leftType, rightValue, rightType, linea, columna)
            "/" -> division(leftValue, leftType, rightValue, rigthT = rightType, linea, columna)
            else -> SintaxError( tipo = "SEMANTICO", mensaje = "Operador No Definido: $operador", linea, columna)
        }
    }

    1 Usage
    private fun suma(leftV: Any?, leftT: TypeData?, rightV: Any?, rightT: TypeData?, linea: Int, columna: Int): Any? {
        //castea todo hacia Double para mayor practisidad
        val a = conversionDouble( unique = leftV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Suma invalida", linea, columna)
        val b = conversionDouble( unique = rightV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Suma invalida", linea, columna)
        val result = a + b //realiza la operacion
        //en caso de ser los valores orginalmente ambos enteros se devuelven como enteros sino como Double
        return if (leftT == TypeData.ENTERO && rightT == TypeData.ENTERO) result.toInt() else result
    }

    1 Usage
    private fun resta(leftV: Any?, leftT: TypeData?, rightV: Any?, rightT: TypeData?, linea: Int, columna: Int): Any? {
        val a = conversionDouble( unique = leftV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Resta inválida", linea, columna)
        val b = conversionDouble( unique = rightV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Resta inválida", linea, columna)
        val result = a - b
        return if (leftT == TypeData.ENTERO && rightT == TypeData.ENTERO) result.toInt() else result
    }

    1 Usage
    private fun multiplicacion(leftV: Any?, leftT: TypeData?, rightV: Any?, rightT: TypeData?, linea: Int, columna: Int): Any? {
        val a = conversionDouble( unique = leftV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Multiplicación inválida", linea, columna)
        val b = conversionDouble( unique = rightV) ?: return SintaxError( tipo = "SEMANTICO", mensaje = "Multiplicación inválida", linea, columna)
        val result = a * b
        return if (leftT == TypeData.ENTERO && rightT == TypeData.ENTERO) result.toInt() else result
    }
```

OperacionMathNoRetornable

Es el encargado de alamcenar las operaciones aritméticas para poder pasarlas en formato a la lista de reportes aritméticos.

A base de del operador ingresado es como maneja las instrucciones, si es +, -, /, * los agrega a la lista aritmética junto con sus operaciones. En caso de no ser dichos valores anteriores únicamente se encarga de unir la condicional para que al momento de guardarla en la lista de las condicionales sea un único texto

```
    override fun interprete(tree: Tree, table: TableSymbol): Any? {
        val leftValue = left.interprete(tree, table)
        if (leftValue is Error)
            return leftValue

        val rightValue = right.interprete(tree, table)
        if(rightValue is Error)
            return rightValue

        val tipoOperador = when (operador){ //cambia los operadores de simbolos a su nombre
            "+" -> "Suma"
            "-" -> "Resta"
            "*" -> "Multiplicacion"
            "/" -> "Division"
            ">=" -> "Mayor Igual Que"
            "<=" -> "Menor Igual Que"
            ">" -> "Mayor Que"
            "<" -> "Menor Que"
            "==" -> "Igual"
            "!=" -> "Diferente"
            else -> null
        }

        //Si el operador es valido y es una operacion aritmetia lo agrega a la lista de aritmeticos
        if (tipoOperador != null && soloAritmeticos( operadorV = tipoOperador) == true){
            //convierte a un solo string la operacion aritmetica completa
            val ocurrencia = " "+valorNumero(leftValue)+" "+operador+" "+valorNumero(rightValue)+" "
            tree.agregarAritmeticos( reporte = Aritmeticos(tipoOperador, linea, columna, ocurrencia))
        }
        //devuelve el valor de la operacion aritmetica
        return when (operador) {
            "+" , "-", "*", "/" -> OperacionesReportes.calcular(leftValue, leftType = left.typeValue.typeData, rightValue, rightType = right.typeValue.typeData, operador,linea, columna)
            else -> true
        }
    }

    //devuelve en formato para la estructura de cotrol
    3 Usages
    fun toTexto():String{
        return "${crearTexto(left)} $operador ${crearTexto(right)}"
    }

    //
    2 Usages
    private fun crearTexto(texto : Instruccion):String{
       return when(texto){
            is AccesVariable -> texto.name //obtiene el nombre del id
            is Native -> texto.value.toString() //si es numero lo pasa a string
            is OperacionMathNoRetornable -> texto.toTexto() //si ya es una estructura de control la devuelve
            else -> " " //no ser valido manda una cadena vacia
        }
```

Native

Únicamente se encarga de alamacenar los valores a su estado mas primitivo, es decir a tipo ENTERO, DECIMAL, STRING y demás tipos necesarios

```
//Para poder almacenar los nombres de los identificadores y valores numericos
22 Usages
class Native (
    val value: Any?,
    type: Type,
    linea: Int,
    columna: Int
) : Instruccion(
    typeValue = type,
    linea,
    columna
){
    override fun interprete(tree: Tree, table: TableSymbol): Any? {
        return value//unicamente devuelve el valor
    }
}
```

Flujos – SI y MIENTRAS

Las estructuas para SI y MIENTRAS tienen la misma forma, por ende únicamente se mostrara el de la estrucuctura SI.

Recibe dos valores, un valor único que seria la condicional y el otro seria un bloque de código que se agrega a una lista

Antes de verificar los valores de la lista se guarda el condicional a la lista de control para poder mostrarlo dentro de los reportes

```
class Estructuira_SI (
     private val unique: Instruccion, //recibe una instrucion para condicional
     private val instrucciones :LinkedList<Instruccion>?, //enlista mas instrucciones para poder almacenarlas dentro de SI
    linea : Int,
    columna : Int
) : Instruccion(
     typeValue = Type(TypeData.VOID),
    linea,
    columna
){
    override fun interprete(tree: Tree, table: TableSymbol): Any? {
        //Si unique llega a ser una estructura de condicion lo almacena para el uso de reporte
        //en caso de no ser una condicion se agrega ¿?
        val condicion = if(unique is OperacionMathNoRetornable) unique.toTexto() else "¿?"
        tree.agregarControl( reporte = Control( objeto = "SI",linea,condicion))

        val uniqueVal = unique.interprete(tree, table)

        if (uniqueVal == true)
            if (instrucciones != null)
                for(instruc in instrucciones) //itera todas las instrucciones almacenadas para guardaras en el ast y ser usadas posteriormente
                    instruc.interprete(tree, table)

        return null //devuelve null para no tener confusion con las condicionales
    }
}
```

Instrucción

Reportes – Aritmeticos y Control

Únicamente son las formas en como serán guardados

```
//clase para poder almacenar los de forma ordenda las operaciones aritmeticas del codigo ingresado
5 Usages
class Aritmeticos (
    val operador : String,
    val linea : Int,
    val columna : Int,
    val ocurrencia : String
){
```

```
3   //clase para poder tener ordenada las estructuras de control encontradas
    7 Usages
4   class Control(
5       val objeto : String,
6       val linea: Int,
7       val condicion : String
8   ) {
9   }
```